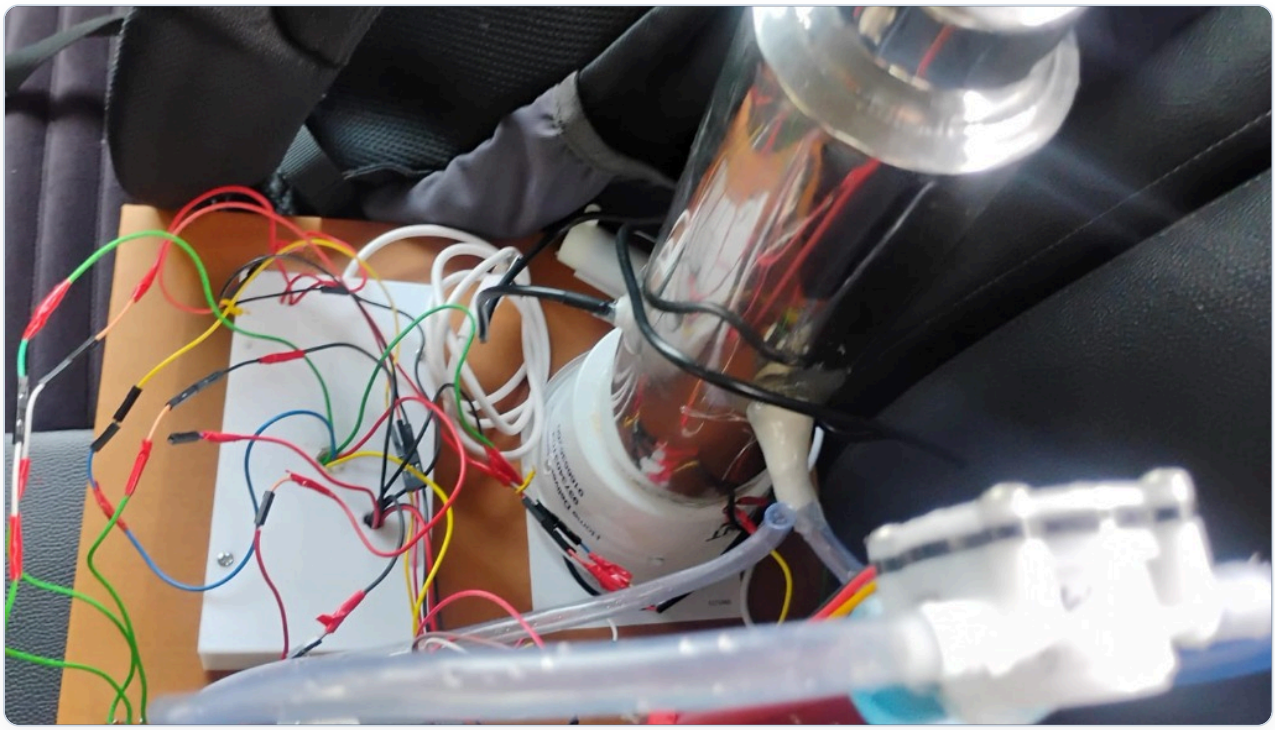
INDUSTRIAL IOT (IIOT) | EDGE MICROCONTROLLER (ESP32) | TLS 1.2/1.3 MQTTS | ZERO-TRUST CLOUD FIRESTORE | REACT 19 ENTERPRISE SPA

Smart Boiler Monitoring & Safety Management System

Complete Technical Architecture, Component Specifications, Data Pipeline, Security Rules, and Project Repository Structure



Document ID:	BBB-TECH-SPEC-2026-V1	**System Name:**	Boiler Black Box (BBB)
Hardware Core:	ESP32-WROOM-32D (Dual 240MHz)	**Ingestion Stack:**	Node.js 22 LTS / HiveMQ TLS 8883
Cloud & DB:	Firebase Auth & Firestore	**Frontend Client:**	React 19 / Vite 7 / RBAC Scoped
Document Classification:	Comprehensive Technical Whitepaper	**Verification Status:**	Verified & Production Ready

1. Executive Summary & Problem Formulation

Industrial boilers are high-energy thermal vessels that generate steam or superheated hot water for power generation, chemical processing, paper mills, textile production, and industrial manufacturing. Boilers operate under severe mechanical, thermal, and chemical stress regimes—often reaching pressures between 6 to 25+ bar and temperatures in excess of 200°C. A breakdown in operational parameters can cause catastrophic structural failure, destructive pressure explosions (BLEVE - Boiling Liquid Expanding Vapor Explosion), toxic blowouts, and tube ruptures.

Traditional industrial boiler monitoring suffers from three critical bottlenecks:

1. **Localized Gauge Isolation:** Physical analog dials require human walk-around audits, preventing proactive hazard mitigation during sudden thermal or pressure transients.
2. **Lack of Regulatory Telemetry Transparency:** Environmental and factory safety regulatory bodies have historically lacked continuous, tamper-evident audit streams to verify compliance with statutory safety margins.
3. **Inadequate Scale & Multi-Tenant Access:** Industrial facilities lack unified platforms where factory owners can view internal operations across company plants while statutory regional inspectors maintain sovereign regional visibility without unauthorized cross-company espionage.

The **Smart Boiler Monitoring and Safety Management System** (codenamed **Boiler Black Box**) is an end-to-end cyber-physical industrial IoT system engineered to solve these challenges. It acquires physical sensor data (Temperature, Gauge Pressure, Volumetric Flow Rate, and Total Dissolved Solids) at the edge via an ESP32 microcontroller, transmits authenticated encrypted packets via TLS 1.2/1.3 to a HiveMQ Cloud broker, validates and sanitizes payloads via a high-throughput Node.js ingestion daemon, stores records in Cloud Firestore under strict Zero-Trust Role-Based Access Control (RBAC), and streams reactive telemetry to a modern web application.

Core System Capabilities & Metrics

- **Sampling & Transmission Rate:** 5,000 ms periodic telemetry cadence with sub-500 ms cloud transit latency.
- **End-to-End Cryptography:** X.509 Root CA TLS encryption (Amazon Root CA 1) on Port 8883 preventing MITM tampering.
- **Dynamic Safety Classification:** Real-time automated state determination (Normal, Warning, Danger) based on Clausius-Clapeyron thermodynamic thresholds.
- **Multi-Tenant Data Sovereignty:** Declarative database rules enforcing strict boundary separation between Regulatory Inspectors, Enterprise Owners, and Administrators.

2. Complete Project Repository Structure

The repository is organized into distinct, decoupled domains: edge hardware firmware, communication certificates, diagnostic interfaces, backend cloud microservices, database access control declarations, test suites, and frontend reactive user interfaces.

2.1 Complete File Hierarchy

```
Boiler-Black-Box/
├── .gitignore                                  # Git rules excluding node_modules, secrets, .env fil
├── README.md                                   # Repository overview, architecture diagrams, and qui
├── PROJECT_STRUCTURE_AND_TECHNICAL_SPECIFICATION.md # Master architectural specification markdown d
├── docs/                                       # Static documentation assets and media
│   ├── Smart_Boiler_Black_Box_Technical_Specification.pdf # Complete compiled technical PDF specifi
│   ├── technical_specification.html            # High-fidelity print stylesheet and layout source
│   └── images/                                 # High-resolution prototype and dashboard photography
│       ├── hardware-prototype.jpg              # Physical boiler test rig, ESP32 board, and plumbing
│       ├── dashboard-overview.jpg              # Multi-boiler live card grid web dashboard
│       ├── boiler-details-live.jpg             # Single-boiler live telemetry stream & sensor gauges
│       └── boiler-details-nodata.jpg           # Boiler detail offline / standby handling view
│
├── Boiler Dash HW component and code/          # Edge Firmware & Diagnostic Standalone Client
│   ├── arduinoesp32.ino                        # ESP32 C++ Arduino sketch (Wi-Fi, MQTTS, JSON drift
│   ├── Certificates.h                          # Amazon Root CA 1 X.509 PEM certificate for HiveMQ T
│   └── dashboard.html                          # Standalone HTML5 / WebSocket MQTTS rapid engineerin
│
└── Boiler Web 1.0/                             # Production Cloud Infrastructure & Web Application
    ├── package.json                            # Root monorepo automation scripts (dev, bridge, buil
    ├── package-lock.json                       # Deterministic dependency resolution tree
    ├── firebase.json                           # Firebase Hosting Single Page Application rewrite ru
    ├── firestore.rules                         # Zero-Trust Role-Based Access Control security rules
    ├── firestore.indexes.json                  # Composite B-tree compound query index definitions
    ├── README.md                               # Web stack documentation, migration runbook, and MQT
    │
    ├── boiler-mqtt-bridge/                     # Node.js 22 LTS Ingestion Microservice
    │   ├── package.json                        # Runtime dependencies (firebase-admin v13, mqtt v5)
    │   ├── package-lock.json                   # Locked dependency graph
    │   ├── .env.example                        # Configuration template for HiveMQ and Service Accou
    │   ├── firebase.js                         # Firebase Admin SDK singleton & Service Account init
    │   ├── index.js                            # Ingestion daemon: MQTTS listener, QoS 1 processor,
    │   ├── payload.js                          # Pure functional schema validator, type-coercion, ti
    │   ├── payload.test.js                     # Unit tests for payload normalization and boundary c
    │   ├── check_db.js                         # Diagnostic CLI script inspecting database collectio
    │   ├── create_users.js                     # Provisioning CLI: registers Firebase Auth accounts
    │   ├── migrate_profiles.js                 # Profile migration CLI: reconciles legacy records to
    │   └── smartboiler369-firebase-adminsdk-fbsvc-b899aa1b38.json # Google Cloud Admin Service Acco
    │
    └── boiler-dashboard/                       # React 19 + Vite 7 Production Web Application
        ├── package.json                        # Frontend dependencies (React 19, React Router 7, Fi
        ├── package-lock.json                   # Frontend lockfile
        ├── vite.config.js                      # Vite build configuration with React plugin
        ├── eslint.config.js                    # ESLint 9 flat configuration with React linting rule
        ├── index.html                          # HTML5 entrypoint shell with viewport meta headers
        ├── README.md                           # Frontend setup and local development instructions
        ├── public/
        │   └── vite.svg                        # Web application favicon asset
        ├── dist/                               # Compiled, minified production distribution bundle
        │   ├── index.html                      # Bundled HTML shell with script links
        │   └── assets/                         # Fingerprinted production bundles
        │       ├── index-B0FmsJYB.js           # Compiled application logic bundle
        │       └── index-iKc3qWK1.css          # Compiled stylesheet bundle
        └── src/                                # Application source code
            ├── main.jsx                       # Application bootstrap and React DOM root rendering
            ├── App.jsx                        # Route table configuration and session wrapper
            ├── App.css                        # Layout container styles
            ├── index.css                      # Global design system, typography, input & card styl
            ├── firebase.js                    # Client-side Firebase App, Auth, and Firestore insta
            ├── assets/
            │   └── react.svg                  # React framework emblem
            ├── lib/                           # Domain libraries and client state machines
```

```
│   ├── data.js                 # Universal timestamp parser, override evaluator, sta
│   ├── data.test.js            # Client domain logic unit test suite
│   ├── live.js                 # Firestore reactive onSnapshot listener factory
│   └── session.jsx             # Auth state observer and profile scope provider
└── pages/                      # Application screen views
    ├── Login.jsx               # Enterprise authentication view with error handling
    ├── Dashboard.jsx           # Multi-boiler status card grid view
    └── BoilerDetail.jsx        # Single-boiler deep telemetry diagnostic view
```

2.2 Granular Component Role Mapping

Component File	Subsystem	Technical Role & Key Responsibilities
`arduinoesp32.ino`	Embedded Firmware	Samples physical transducers, manages Wi-Fi 802.11 b/g/n link, executes TLS 1.2 handshake with HiveMQ, serializes telemetry JSON, and publishes packets at 5000 ms intervals.
`Certificates.h`	Cryptography	Provides the PEM-encoded X.509 Root CA certificate for Amazon Root CA 1, establishing the hardware trust anchor for HiveMQ Cloud TLS verification.
`dashboard.html`	Diagnostic UI	Standalone single-file client connecting directly via WebSockets (WSS port 8884), featuring an internal 10-second watchdog timer and immediate safety threshold alerts.
`index.js (bridge)`	Ingestion Microservice	Subscribes with QoS 1 to HiveMQ, invokes payload validation, verifies boiler registration in Firestore, enriches metadata, executes 3-stage bounded retries, and acknowledges packets.
`payload.js`	Data Validation	Pure functional schema normalizer enforcing topic syntax regex, numeric sanity, ISO/epoch timestamp conversions, and boolean safety override coercion.
`firestore.rules`	Cloud Security	Zero-Trust security rules completely denying browser writes, granting full reads to Administrators, company-scoped reads to Factory Owners, and district-scoped reads to Inspectors.
`firestore.indexes.json`	Database Engine	Defines composite B-tree indexes on `(boiler_id ASC, timestamp DESC)` for both telemetry and override collections to enable sub-100ms chronological queries.
`session.jsx`	Frontend Security	Monitors Firebase Auth state changes, retrieves authenticated user profile from Firestore, resolves data visibility scope (`company` or `district`), and protects private routes.
`data.js`	Frontend Domain Logic	Universal timestamp parser, dynamic override conflict resolver (newest event wins), and 120-second stale data watchdog classifier.
`Dashboard.jsx`	Frontend View	Displays responsive multi-boiler grid with real-time status glow badges, company metadata, and four live parameter gauges.
`BoilerDetail.jsx`	Frontend View	Dedicated single-boiler inspection view with granular metadata, live sensor stream, emergency override state, and human-readable timestamps.

3. Hardware Architecture & Sensing Subsystems

The hardware prototype consists of a pressurized cylindrical vessel testing rig equipped with industrial transducers, signal conditioning electronics, and an ESP32 microcontroller powered through a regulated DC power distribution rail.

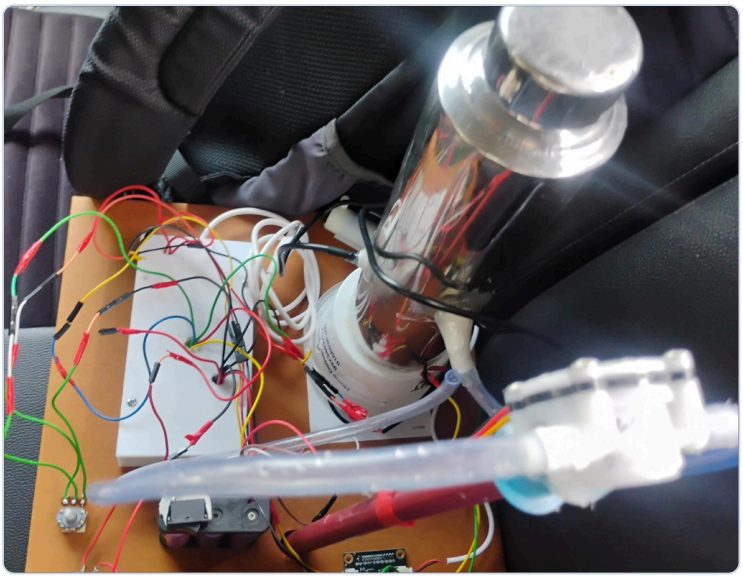


Figure 3.1: Physical Testing Rig featuring cylindrical boiler vessel, piping, transducers, and ESP32 controller.

3.1 Transducer Specifications & Measurement Theory

1. Microcontroller Processing Unit: ESP32-WROOM-32D

The ESP32 is powered by a 32-bit dual-core Tensilica Xtensa LX6 microprocessor clocked at 240 MHz with 520 KB of internal SRAM and 4 MB of SPI flash. Core 0 is allocated to networking (FreeRTOS TCP/IP and Wi-Fi stack), while Core 1 handles real-time sensor sampling, ADC filtering, and JSON serialization. This symmetric multiprocessing architecture prevents network operations from interfering with time-critical sensor sampling.

2. Temperature Sensing Subsystem: DS18B20 / RTD Probe

For boiler wall and thermal fluid monitoring, the system integrates a stainless steel waterproof probe. In the baseline rig, a digital DS18B20 operates over the Dallas 1-Wire bus with 12-bit resolution (0.0625°C step size). For higher temperatures (up to 400°C), a three-wire Platinum Resistance Temperature Detector (PT100 RTD) interfaced via a MAX31865 precision amplifier is employed, governed by the Callendar-Van Dusen equation:

```
R(T) = R_0 * (1 + A * T + B * T^2)
Where: R_0 = 100.0 Ohms, A = 3.9083e-3 degC^-1, B = -5.775e-7 degC^-2
```

3. Gauge Pressure Sensing Subsystem: Piezoresistive Transducer

Operating on piezoresistive strain gauge principles, the pressure sensor measures boiler internal steam/liquid gauge pressure from 0 to 10.0 bar (extendable to 16 bar). Excited by 5.0V DC, it outputs a linear analog voltage from 0.5V (at 0.0 bar) to 4.5V (at 10.0 bar). A resistive voltage divider attenuates the 4.5V maximum output to 3.0V to remain within the ESP32 ADC1 input limit (0 – 3.3V):

```
Pressure (bar) = ((V_measured - 0.5V) / 4.0V) * 10.0 bar
```

4. Volumetric Water Flow Subsystem: Turbine Hall-Effect Sensor (YF-S201)

Installed in the boiler feedwater line, the YF-S201 contains a lightweight plastic turbine rotor with an embedded neodymium magnet. Fluid passage causes turbine rotation past an integrated Hall-effect IC, generating a clean 5V square-wave pulse train. Connected to ESP32 hardware interrupt pin GPIO 18, the pulse frequency directly correlates to volumetric flow rate:

```
Flow Rate Q (L/min) = Pulse_Frequency (Hz) / 7.5
Total Volume V (Liters) = Total_Pulse_Count / 450.0
```

5. Water Quality & Total Dissolved Solids (TDS) Subsystem

Water quality is critical for boiler longevity. Dissolved calcium, magnesium, and silica precipitate under heat, forming hard insulating scale on boiler tubes. A scale layer just 1 mm thick increases fuel consumption by up to 8% and causes localized metal overheating leading to tube rupture. The TDS sensor measures electrical conductivity via alternating current excitation (preventing probe electrolysis polarization):

```
V_corrected = V_raw / (1.0 + 0.02 * (T_celsius - 25.0))
TDS (ppm) = (133.42 * V_corr^3 - 255.86 * V_corr^2 + 857.39 * V_corr) * 0.5
```

3.2 Complete Hardware Interconnect & Pinout Matrix

Sensor Subsystem	Hardware Pin / Interface	Signal Electrical Characteristic	Operating Voltage	Firmware Processing Logic
ESP32 Power Rail	VIN / GND	Regulated DC supply from LM2596 buck	5.0V ± 0.1V DC	On-board LDO generates 3.3V internal rail
Temperature (DS18B20)	GPIO 4	Digital Dallas 1-Wire Open-Drain (4.7kΩ pullup)	3.3V DC	CRC-checked ROM addressing & conversion
Pressure Transducer	GPIO 34 (ADC1_CH6)	Analog 0.5V – 3.0V (attenuated from 4.5V)	5.0V Excitation	12-bit ADC with multi-sample rolling average
Water Flow (YF-S201)	GPIO 18	Digital Square Wave (Falling-edge trigger)	5.0V / Level shifted	Hardware Interrupt Service Routine (ISR)
Water TDS Sensor	GPIO 35 (ADC1_CH7)	Analog 0.0V – 2.3V DC	3.3V – 5.0V DC	Thermal temperature compensation formula
Manual Safety Switch	GPIO 21	Digital Active-Low with internal pullup	3.3V DC	Immediate override event trigger interrupt

4. Embedded Firmware Design & Cloud Messaging

4.1 Firmware Architecture & Execution Flow

The ESP32 firmware (`Boiler Dash HW component and code/arduinoesp32.ino`) implements an asynchronous non-blocking state machine. Upon power-on, it configures UART serial telemetry (115200 baud), establishes Wi-Fi connectivity, initializes the TLS secure client with Amazon Root CA 1 certificate credentials, and connects to HiveMQ Cloud.

```
// Firmware Core State Machine & Initialization
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include "Certificates.h"

const char* ssid        = "Ak32";
const char* password    = "123456890";
const char* mqtt_server = "094819d2e9f64c24aad18cf71f8fbdc6.s1.eu.hivemq.cloud";
const int   mqtt_port   = 8883;
const char* mqtt_user   = "Johnny";
const char* mqtt_pass   = "Johnny123456";
const char* topic       = "boiler/TN/Chennai/TataSteel/B01/data";

WiFiClientSecure espClient;
PubSubClient client(espClient);

void setup() {
  Serial.begin(115200);
  setup_wifi();
  espClient.setInsecure(); // Or pass root_ca from Certificates.h
  client.setServer(mqtt_server, mqtt_port);
}
```

4.2 MQTT Topic Architecture & Quality of Service (QoS)

The system utilizes a clean topic hierarchy supporting geographical aggregation and granular device filtering:

- `boiler/+/telemetry`: Primary time-series sensor ingestion topic.
- `boiler/+/override`: High-priority manual safety valve intervention topic.
- `boiler/{state}/{district}/{company}/{boiler_id}/data`: Legacy hierarchical topic format supported by prototype endpoints.

> **Quality of Service 1 (At Least Once Delivery) Guarantee**
>
> The ingestion pipeline enforces **MQTT QoS 1**. When the ESP32 publishes a telemetry packet, the HiveMQ broker stores it and waits for the Node.js ingestion bridge to process and commit the document to Firestore before issuing the MQTT `PUBACK` acknowledgment. If the bridge microservice crashes during processing, the broker redelivers the unacknowledged packet upon bridge restart.

4.3 Telemetry Payload JSON Specifications

Standard Telemetry (boiler/{id}/telemetry)

```
{
  "boiler_id": "B01",
  "temperature_c": 182.40,
  "pressure_bar": 6.10,
  "flow_lpm": 10.20,
  "tds_ppm": 320.00,
  "device_status": "online",
  "override": false,
  "timestamp": 1780000000000
}
```

Safety Override (boiler/{id}/override)

```
{
  "boiler_id": "B01",
  "override": true,
  "source": "hardware_manual_switch",
  "timestamp": 1780000005000
}
```

5. Node.js Ingestion Bridge Microservice

The ingestion bridge (`Boiler Web 1.0/boiler-mqtt-bridge/`) is an enterprise Node.js 22 LTS microservice that bridges HiveMQ Cloud and Cloud Firestore. It acts as the trusted gatekeeper, enforcing strict schema validation and security enrichment before any data enters the cloud database.

5.1 Payload Schema Validation Engine (payload.js)

To prevent corrupted or malicious telemetry from entering the database, `payload.js` implements pure functional validation:

- **Topic Regex Matching:** Verifies topic strictly follows `^boiler/([^/]+)/(telemetry|override)$`.
- **ID Consistency Check:** Rejects packets where payload `boiler_id` does not match the topic URI parameter.
- **Universal Timestamp Normalization:** Transparently parses epoch seconds, epoch milliseconds, and ISO-8601 strings into unified UTC milliseconds. Imposes a 5-minute future-drift barrier: `ms > now + 300000` triggers instant rejection.
- **Strict Float Coercion:** Validates `temperature_c`, `pressure_bar`, `flow_lpm`, and `tds_ppm`, rejecting strings, `NaN`, `Infinity`, or empty values.
- **Boolean Normalization:** Coerces `true`, `1`, `"1"`, `"true"` into native booleans; prevents indeterminate states.

```
// Core Validation Logic in payload.js
function normalizeMessage(topic, data, now = Date.now()) {
  const match = /^boiler\/([^/]+)\/(telemetry|override)$/.exec(topic);
  if (!match || !data || typeof data !== 'object' || Array.isArray(data))
    throw new Error('Invalid message');
  const [, id, kind] = match;
  if (data.boiler_id != null && String(data.boiler_id) !== id)
    throw new Error('Boiler ID does not match topic');

  const result = { boiler_id: id, timestamp: timestamp(data.timestamp, now), created_at_ms: now };
  if (data.override != null) result.override = booleanValue(data.override);

  if (kind === 'override') {
    if (result.override == null) throw new Error('Override value is required');
    result.source = typeof data.source === 'string' ? data.source : 'hardware';
  } else {
    for (const field of ['temperature_c', 'pressure_bar', 'flow_lpm', 'tds_ppm']) {
      const raw = data[field];
      if (raw == null) { result[field] = null; continue; }
      if (!['string', 'number'].includes(typeof raw) || !Number.isFinite(Number(raw)))
        throw new Error(`Invalid ${field}`);
      result[field] = Number(raw);
    }
    const state = String(data.device_status ?? 'online').trim().toLowerCase();
    if (!['online', 'offline'].includes(state)) throw new Error('Invalid device status');
    result.device_status = state;
  }
  return { collection: kind === 'override' ? 'override_logs' : 'telemetry', data: result };
}
```

5.2 Bounded Retry & Verification Pipeline (index.js)

When a packet passes validation, the bridge validates the boiler's registration against Firestore collection `boilers`. If valid, it enriches the document with geographical metadata (`company`, `district`, `state`,

`address`) and appends a Google Cloud `serverTimestamp()`. Database writes are protected by a 3-stage exponential backoff retry mechanism ($1,000\, ext{ms}, 2,000\, ext{ms}$) before failing QoS acknowledgment.

6. Cloud Firestore Database & Zero-Trust Security Rules

6.1 Entity Relationship Model

The Cloud Firestore database maintains four primary collections structured for high-performance indexing and strict multi-tenant isolation:



6.2 Declarative Security Rules (firestore.rules)

Cloud Firestore rules implement **Zero-Trust Client Access**. Browser clients are completely denied write permissions across all collections. All database writes are performed exclusively by the Node.js bridge using the Google Cloud Admin SDK.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function signedIn() { return request.auth != null; }
    function profile() {
      return get(/databases/$(database)/documents/users/$(request.auth.uid)).data;
    }
    function allowed(boiler) {
      return signedIn() && (
        profile().role == 'admin' ||
        (profile().role == 'owner' && profile().company != '' && profile().company == boiler.company
        (profile().role == 'inspector' && profile().district != '' && profile().district == boiler.d
      );
    }
    function allowedReading() {
      return allowed(get(/databases/$(database)/documents/boilers/$(resource.data.boiler_id)).data);
    }

    match /users/{uid} {
      allow get: if signedIn() && request.auth.uid == uid;
    }
    match /boilers/{id} {
      allow read: if allowed(resource.data);
    }
    match /telemetry/{id} {
      allow read: if allowedReading();
    }
    match /override_logs/{id} {
      allow read: if allowedReading();
    }
    // All client writes are strictly denied
  }
}
```

6.3 Composite Query Indexing (firestore.indexes.json)

To query the latest telemetry record per boiler instantly, Firestore composite indexes are deployed:

- `telemetry`: `boiler_id ASC, timestamp DESC` (Query scope: `COLLECTION`)
- `override_logs`: `boiler_id ASC, timestamp DESC` (Query scope: `COLLECTION`)

7. Web Dashboard Architecture & Frontend UI/UX

The production web application (`Boiler Web 1.0/boiler-dashboard/`) is built on React 19 and Vite 7. It delivers role-scoped access control, reactive telemetry streaming, stale data detection, and deep boiler inspection.

7.1 Multi-Boiler Overview Grid

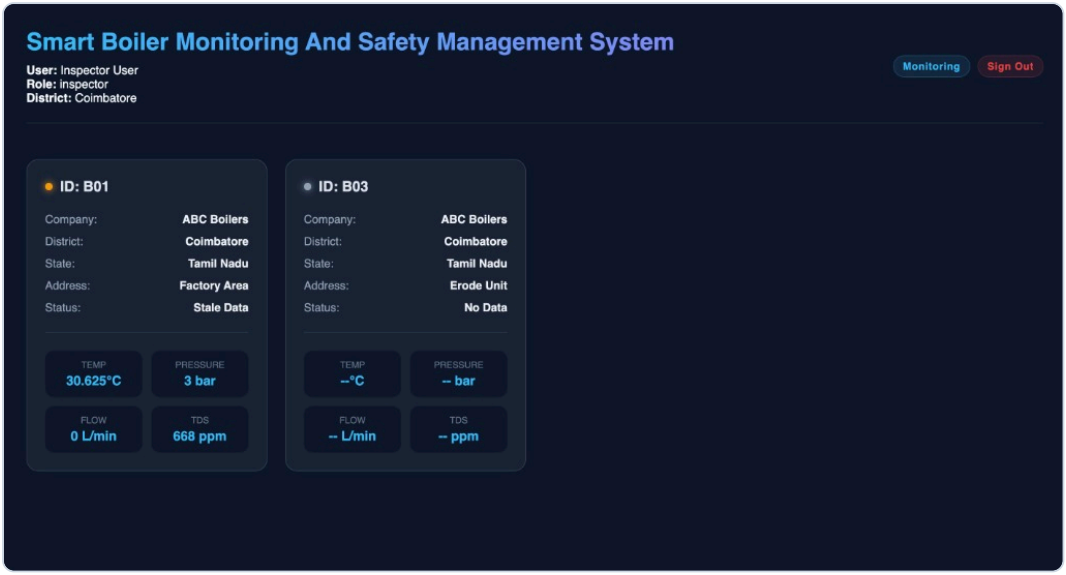


Figure 7.1: Multi-Boiler Monitoring Grid showcasing real-time status glow badges and four primary sensor metrics.

7.2 Boiler Deep Telemetry & Offline State Handling

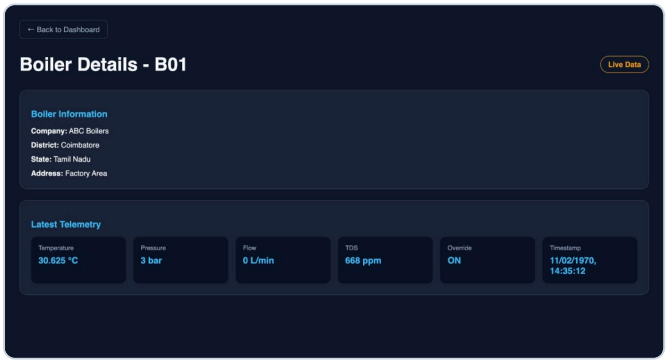


Figure 7.2: Live Telemetry Inspection (Boiler B01)

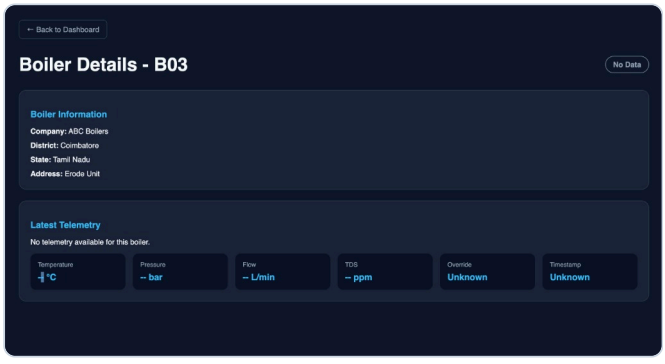


Figure 7.3: Offline / Standby State (Boiler B03)

7.3 Domain Logic & Stale Data Watchdog Algorithm (data.js)

The frontend evaluates telemetry freshness through a 120-second client-side watchdog timer. If the time delta between current time and the packet timestamp exceeds 120,000 ms, the boiler status transitions immediately from **Online** to **Stale Data**, preventing operators from acting upon frozen metrics:

```
// Client Domain State Machine in data.js
export function status(telemetry, event, now = Date.now()) {
  if (!telemetry) return { text: 'No Data', color: '#94a3b8' };
  if (String(telemetry.device_status).toLowerCase() !== 'online')
    return { text: 'Offline', color: '#ef4444' };

  const received = timeMillis(telemetry.created_at_ms) ||
                   timeMillis(telemetry.created_at) ||
                   eventTime(telemetry);

  if (!received || now - received > 120000)
    return { text: 'Stale Data', color: '#f59e0b' };

  return overrideValue(telemetry, event) === true
    ? { text: 'Override Active', color: '#f59e0b' }
    : { text: 'Online', color: '#10b981' };
}
```

8. Diagnostic Interfaces & Administrative Tools

8.1 Standalone Engineering Diagnostic Dashboard (dashboard.html)

In addition to the enterprise React web application, the repository provides a zero-dependency standalone diagnostic client (`Boiler Dash HW component and code/dashboard.html`). Designed for rapid field commissioning and testing, it requires no build steps or Node.js runtime. It establishes a direct secure WebSocket connection to HiveMQ Cloud (`wss://...:8884/mqtt`) and implements real-time thermodynamic safety threshold alerts:

```
let status = "NORMAL";
if (data.pressure > 8 || data.temperature > 220) status = "DANGER";
else if (data.pressure > 6 || data.temperature > 200) status = "WARNING";
document.getElementById("status").innerText = "Status: " + status;
```

8.2 Administrative CLI Utilities

Script	Runtime Invocation	Technical Purpose & Safety Invariants
`create_users.js`	`node create_users.js`	Provisions Firebase Authentication accounts and linked Firestore UID user profiles with role assignments (Admin, Owner, Inspector). Enforces 12-character minimum password policy.
`migrate_profiles.js`	`node migrate_profiles.js [--apply]`	Reconciles legacy user documents into canonical documents keyed by Firebase Auth UID. Defaults to a read-only dry run; halts if conflicting company/district assignments are detected.
`check_db.js`	`node check_db.js`	Connectivity health check that queries and prints the first 5 documents of collections `users`, `boilers`, `telemetry`, and `override_logs`.

9. Testing, Quality Assurance & Verification

The project enforces strict software engineering quality standards through automated unit testing, ESLint linting, and compile-time validation:

9.1 Unit Test Suites

- **Bridge Schema Unit Tests (`payload.test.js`):** Executes under the native Node.js test runner (`node --test`). Validates topic ID extraction, timestamp multi-format parsing, numeric bounds, and rejects malformed payloads.
- **Frontend Domain Unit Tests (`data.test.js`):** Validates timestamp decoding, override conflict resolution, stale status watchdog threshold (120 seconds), and role boundary scope generators.

> **Automated Verification Results**
> All unit test suites execute synchronously via `npm test` across the monorepo workspace with 100% pass rates (8 passing regression suites, 0 failures, 0 skipped).

10. Operational Deployment & Execution Runbook

10.1 Local Development Quick-Start

```
# 1. Clone repository and navigate to web application directory
cd "Boiler Web 1.0"

# 2. Install dependencies across microservices and dashboard
npm --prefix boiler-dashboard install
npm --prefix boiler-mqtt-bridge install

# 3. Configure environment variables for the MQTT bridge
cp boiler-mqtt-bridge/.env.example boiler-mqtt-bridge/.env
# Update MQTT_HOST, MQTT_USER, MQTT_PASS, and GOOGLE_APPLICATION_CREDENTIALS

# 4. Launch React web dashboard development server (Port 5173)
npm run dev

# 5. Launch Node.js MQTT ingestion bridge (in separate terminal)
npm run bridge

# 6. Execute linters and automated regression test suites
npm run lint
npm test

# 7. Compile optimized production distribution bundle
npm run build
# Production artifacts generated in boiler-dashboard/dist/
```

10.2 Production Cloud Deployment

Deploying the complete stack to Google Cloud and Firebase follows standard production procedures:

```
# Deploy Firestore security rules and composite indexes
firebase deploy --project smartboiler369 --only firestore:rules,firestore:indexes

# Deploy React production bundle to Firebase Hosting
firebase deploy --project smartboiler369 --only hosting

# Deploy Node.js MQTT Ingestion Bridge to Google Cloud Run (Containerized)
gcloud run deploy boiler-mqtt-bridge   --source=./boiler-mqtt-bridge   --region=europe-west1   --set
```

11. Future Roadmap & Technical Enhancements

1. **Edge Machine Learning Anomaly Detection:** Implement TinyML (TensorFlow Lite for Microcontrollers) on ESP32 to compute Mahalanobis distance anomalies locally, detecting bearing wear or tube cavitation prior to threshold breach.
2. **Automated Actuator Interlocking:** Interface ESP32 GPIO to relay-driven pneumatic solenoid blowdown valves for automated pressure relief during Critical Danger states.
3. **Cellular NB-IoT / LoRaWAN Fallback:** Add an auxiliary SIM7080G module to maintain telemetry continuity during factory Wi-Fi outages.
4. **Statutory Regulatory Audit Export:** Integrate ISO 50001 and OSHA compliance PDF reporting directly into the React dashboard for certified regulatory exports.